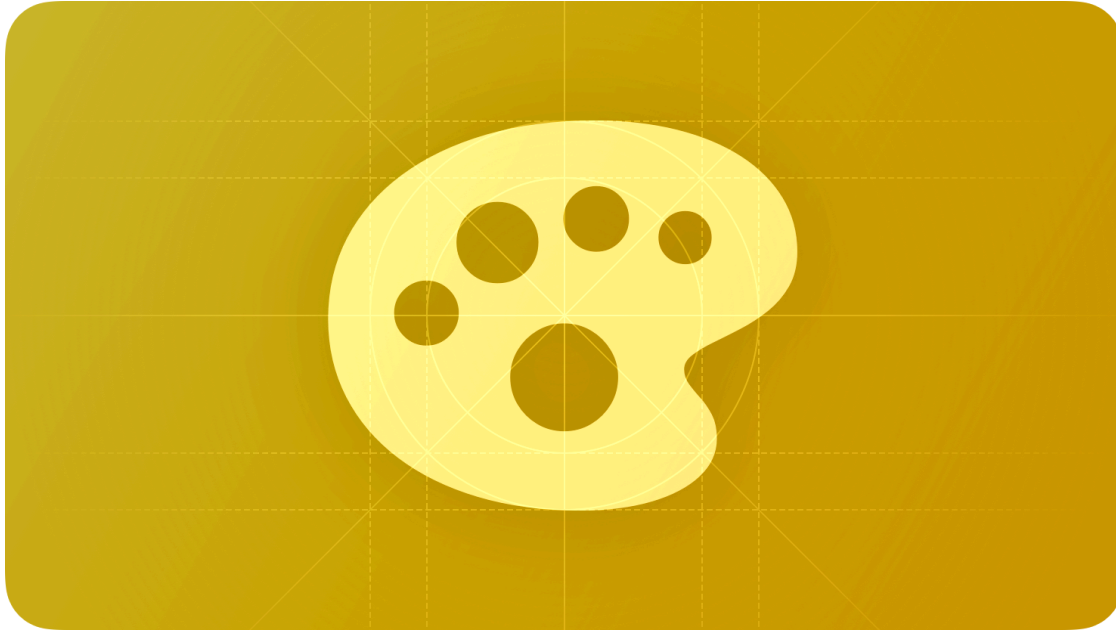


Color

Judicious use of color can enhance communication, evoke your brand, provide visual continuity, communicate status and feedback, and help people understand information.



The system defines colors that look good on various backgrounds and appearance modes, and can automatically adapt to vibrancy and accessibility settings. Using system colors is a convenient way to make your experience feel at home on the device.

You may also want to use custom colors to enhance the visual experience of your app or game and express its unique personality. The following guidelines can help you use color in ways that people appreciate, regardless of whether you use system-defined or custom colors.

Best practices

Avoid using the same color to mean different things. Use color consistently throughout your interface, especially when you use it to help communicate information like status or interactivity. For example, if you use your brand color to indicate that a borderless button is interactive, using the same or similar color to stylize noninteractive text is confusing.

Make sure all your app's colors work well in light, dark, and increased contrast contexts. iOS, iPadOS, macOS, and tvOS offer both light and dark appearance settings. System colors vary subtly depending on the system appearance, adjusting to ensure proper color differentiation and contrast for text, symbols, and other elements. With the Increase Contrast setting turned on, the color differences become far more apparent. When possible, use system colors, which already define variants for all these contexts. If you define a custom color, make sure to supply light and

Supported platforms



- Color
- Best practices
- Inclusive color
- System colors
- Liquid Glass color
- Color management
- Platform considerations
- Specifications
- Resources
- Change log

dark variants, and an increased contrast option for each variant that provides a significantly higher amount of visual differentiation. Even if your app ships in a single appearance mode, provide both light and dark colors to support Liquid Glass adaptivity in these contexts.

Default (light)

Increased contrast (light)

Default (dark)

Increased contrast (dark)

Test your app’s color scheme under a variety of lighting conditions. Colors can look different when you view your app outside on a sunny day or in dim light. In bright surroundings, colors look darker and more muted. In dark environments, colors appear bright and saturated. In visionOS, colors can look different depending on the colors of a wall or object in a person’s physical surroundings and how it reflects light. Adjust app colors to provide an optimal viewing experience in the majority of use cases.

Test your app on different devices. For example, the True Tone display — available on certain iPhone, iPad, and Mac models — uses ambient light sensors to automatically adjust the white point of the display to adapt to the lighting conditions of the current environment. Apps that primarily support reading, photos, video, and gaming can strengthen or weaken this effect by specifying a white point adaptivity style (for developer guidance, see [UIWhitePointAdaptivityStyle](#)). Test tvOS apps on multiple brands of HD and 4K TVs, and with different display settings. You can also test the appearance of your app using different color profiles on a Mac — such as P3 and Standard RGB (sRGB) — by choosing a profile in System Settings > Displays. For guidance, see [Color management](#).

Consider how artwork and translucency affect nearby colors. Variations in artwork sometimes warrant changes to nearby colors to maintain visual continuity and prevent interface elements from becoming overpowering or underwhelming. Maps, for example, displays a light color

scheme when in map mode but switches to a dark color scheme when in satellite mode. Colors can also appear different when placed behind or applied to a translucent element like a toolbar.

If your app lets people choose colors, prefer system-provided color controls where available. Using built-in color pickers provides a consistent user experience, in addition to letting people save a set of colors they can access from any app. For developer guidance, see [ColorPicker](#).

Inclusive color

Avoid relying solely on color to differentiate between objects, indicate interactivity, or communicate essential information. When you use color to convey information, be sure to provide the same information in alternative ways so people with color blindness or other visual disabilities can understand it. For example, you can use text labels or glyph shapes to identify objects or states.

Avoid using colors that make it hard to perceive content in your app. For example, insufficient contrast can cause icons and text to blend with the background and make content hard to read, and people who are color blind might not be able to distinguish some color combinations. For guidance, see [Accessibility](#).

Consider how the colors you use might be perceived in other countries and cultures. For example, red communicates danger in some cultures, but has positive connotations in other cultures. Make sure the colors in your app send the message you intend.



Green indicates a positive trend in the Stocks app in English.



Red indicates a positive trend in the Stocks app in Chinese.

System colors

Avoid hard-coding system color values in your app. Documented color values are for your reference during the app design process. The actual color values may fluctuate from release to release, based on a variety of environmental variables. Use APIs like [Color](#) to apply system colors.

iOS, iPadOS, macOS, and visionOS also define sets of *dynamic system colors* that match the color schemes of standard UI components and automatically adapt to both light and dark contexts. Each dynamic color is semantically defined by its purpose, rather than its appearance or color values. For example, some colors represent view backgrounds at different levels of hierarchy and other colors represent foreground content, such as labels, links, and separators.

Avoid redefining the semantic meanings of dynamic system colors. To ensure a consistent experience and ensure your interface looks great when the appearance of the platform changes,

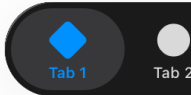
use dynamic system colors as intended. For example, don't use the separator color as a text color, or secondary text label color as a background color.

Liquid Glass color

By default, Liquid Glass has no inherent color, and instead takes on colors from the content directly behind it. You can apply color to some Liquid Glass elements, giving them the appearance of colored or stained glass. This is useful for drawing emphasis to a specific control, like a primary call to action, and is the approach the system uses for prominent button styling. Symbols or text labels on Liquid Glass controls can also have color.



Controls can use color in the Liquid Glass background, like in a primary action button.



Symbols and text that appear on Liquid Glass can have color, like in a selected tab bar item.



By default, Liquid Glass picks up the color from the content layer behind it.

For smaller elements like toolbars and tab bars, the system can adapt Liquid Glass between a light and dark appearance in response to the underlying content. By default, symbols and text on these elements follow a monochromatic color scheme, becoming darker when the underlying content is light, and lighter when it's dark. Liquid Glass appears more opaque in larger elements like sidebars to preserve legibility over complex backgrounds and accommodate richer content on the material's surface.

Apply color sparingly to the Liquid Glass material, and to symbols or text on the material. If you apply color, reserve it for elements that truly benefit from emphasis, such as status indicators or primary actions. To emphasize primary actions, apply color to the background rather than to symbols or text. For example, the system applies the app accent color to the background in prominent buttons — such as the Done button — to draw attention and elevate their visual prominence. Refrain from adding color to the background of multiple controls.

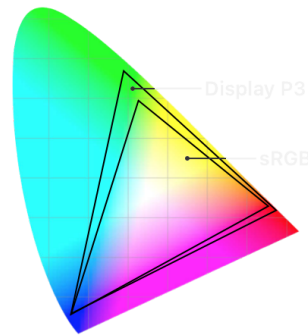


Avoid using similar colors in control labels if your app has a colorful background. While color can make apps more visually appealing, playful, or reflective of your brand, too much color can be overwhelming and make control labels more difficult to read. If your app features colorful backgrounds or visually rich content, prefer a monochromatic appearance for toolbars and tab bars, or choose an accent color with sufficient visual differentiation. By contrast, in apps with primarily monochromatic content or backgrounds, choosing your brand color as the app accent color can be an effective way to tailor your app experience and reflect your company's identity.

Be aware of the placement of color in the content layer. Make sure your interface maintains sufficient contrast by avoiding overlap of similar colors in the content layer and controls when possible. Although colorful content might intermittently scroll underneath controls, make sure its default or resting state — like the top of a screen of scrollable content — maintains clear legibility.

Color management

A *color space* represents the colors in a *color model* like RGB or CMYK. Common color spaces — sometimes called *gamuts* — are sRGB and Display P3.



A *color profile* describes the colors in a color space using, for example, mathematical formulas or tables of data that map colors to numerical representations. An image embeds its color profile so that a device can interpret the image's colors correctly and reproduce them on a display.

Apply color profiles to your images. Color profiles help ensure that your app's colors appear as intended on different displays. The sRGB color space produces accurate colors on most displays.

Use wide color to enhance the visual experience on compatible displays. Wide color displays support a P3 color space, which can produce richer, more saturated colors than sRGB. As a result, photos and videos that use wide color are more lifelike, and visual data and status indicators that use wide color can be more meaningful. When appropriate, use the Display P3 color profile at 16 bits per pixel (per channel) and export images in PNG format. Note that you need to use a wide color display to design wide color images and select P3 colors.

Provide color space–specific image and color variations if necessary. In general, P3 colors and images appear fine on sRGB displays. Occasionally, it may be hard to distinguish two very similar P3 colors when viewing them on an sRGB display. Gradients that use P3 colors can also sometimes appear clipped on sRGB displays. To avoid these issues and to ensure visual fidelity on both wide color and sRGB displays, you can use the asset catalog of your Xcode project to provide different versions of images and colors for each color space.